

Chapitre II

Objectifs

Les objectifs de ce projet sont similaires à ceux de l'ensemble de la première année : rigueur, utilisation du langage C, algorithmique de base, recherche d'informations, etc.

En tant que projet de graphisme, cub3D vous permettra d'améliorer vos compétences dans des domaines tels que la gestion des fenêtres, les couleurs, les événements, le remplissage de formes, etc.

En somme, cub3D constitue un terrain de jeu idéal pour explorer les applications pratiques et ludiques des mathématiques, sans avoir à en maîtriser toutes les subtilités théoriques.

Grâce aux nombreuses ressources disponibles sur Internet, vous utiliserez les mathématiques comme un outil pour concevoir des algorithmes élégants et efficaces.

Si cela vous convient, vous pouvez tester le jeu original avant de commencer ce projet :

<http://users.atw.hu/wolf3d/>

Chapitre III

Instructions communes

- Votre projet doit être **écrit en C**.
- Votre projet doit **respecter la Norme**. Si vous avez des fichiers ou des fonctions bonus, ils sont inclus dans la vérification de la norme ; vous recevrez la note de 0 en cas d'erreur de norme.
- Vos fonctions ne doivent **pas quitter de manière inattendue** (segmentation fault, bus error, double free, etc.), sauf en cas de comportement indéfini. Si cela se produit, votre projet sera considéré comme non fonctionnel et recevra la note de 0 lors de l'évaluation.
- Toute mémoire allouée sur le tas doit être correctement libérée lorsque cela est nécessaire. Les **fuites de mémoire** ne seront **pas tolérées**.
- Si le sujet l'exige, vous devez **fournir un Makefile** qui compile vos fichiers sources pour obtenir la sortie requise avec les flags -Wall, -Wextra et -Werror, en utilisant cc. De plus, votre Makefile ne doit **pas** effectuer **de relinkage** inutile.
- Votre Makefile doit contenir au moins les règles (**NAME**), **all**, **clean**, **fclean** et **re**.
- Pour soumettre des bonus pour votre projet, vous devez inclure une règle bonus dans votre Makefile ; celle-ci ajoutera les différents headers, bibliothèques ou fonctions qui ne sont pas autorisés dans la partie principale du projet. Les bonus doivent être placés dans des fichiers `_bonus.{c/h}`, sauf indication contraire du sujet. L'évaluation des parties obligatoire et bonus est effectuée séparément.
- Si votre projet vous permet d'utiliser votre `libft`, vous devez copier ses sources ainsi que son `Makefile` associé dans un dossier `libft`. Le `Makefile` de votre projet doit compiler la bibliothèque en utilisant son propre `Makefile`, puis compiler le projet.
- Nous vous encourageons à créer des programmes de test pour votre projet, même si **ce travail n'a pas besoin d'être rendu et ne sera pas noté**. Cela vous permettra de tester facilement votre travail ainsi que celui de vos pairs. Ces tests vous seront particulièrement utiles lors de la soutenance ; en effet, vous serez libre d'utiliser vos propres tests et/ou ceux du pair que vous évaluez.
- Déposez votre travail dans le dépôt Git qui vous a été attribué. Seul le contenu du dépôt Git sera pris en compte pour la notation. Si Deepthought est chargé d'évaluer votre travail, cette étape aura lieu après les évaluations par les pairs. Si une erreur survient dans une partie quelconque de votre travail lors de l'évaluation par Deepthought, le processus d'évaluation s'arrêtera.

Chapitre V

Partie obligatoire - cub3D

Nom du programme :	cub3D
Fichiers à rendre :	Tous vos fichiers
Makefile :	all, clean, fclean, re, bonus
Arguments :	une carte au format *.cub
Fonctions externes	• open, close, read, write, printf, malloc, free, perror, strerror, exit, gettimeofday. • Toutes les fonctions de la bibliothèque mathématique (-lm man 3 math). • gettimeofday() • Toutes les fonctions de la bibliothèque MinilibX.
Libft autorisée :	Oui
Description :	Vous devez créer une représentation graphique 3D « réaliste » de l'intérieur d'un labyrinthe, en vue à la première personne. Vous devez réaliser cette représentation en utilisant les principes de *ray-casting* mentionnés précédemment.

Les contraintes sont les suivantes :

- Vous devez utiliser la miniLibX, soit la version disponible sur le système d'exploitation, soit celle issue des sources. Si vous choisissez d'utiliser les sources, vous devrez appliquer à votre libft les mêmes règles que celles énoncées ci-dessus dans la section « Instructions communes ».
- La gestion de la fenêtre doit rester fluide (passage à une autre fenêtre, réduction, etc.).
- Affichez différentes textures de murs (au choix) variant selon l'orientation de la face du mur (Nord, Sud, Est, Ouest).
- Votre programme doit permettre de définir des couleurs différentes pour le sol et le plafond.
- Le programme affiche l'image dans une fenêtre et respecte les règles suivantes :
 - Les touches fléchées gauche et droite du clavier doivent permettre de regarder à gauche et à droite dans le labyrinthe.
 - Les touches **W**, **A**, **S** et **D** doivent permettre de déplacer le point de vue dans le labyrinthe.
 - Appuyer sur la touche Échap (**ESC**) doit fermer la fenêtre et quitter le programme proprement.
 - Cliquer sur la **croix rouge** du cadre de la fenêtre doit fermer la fenêtre et quitter le programme proprement.
 - L'utilisation des fonctions de gestion d'images de la bibliothèque minilibX est vivement recommandée.
- Votre programme doit prendre comme premier argument un fichier de description de scène avec l'extension **.cub**.
- La carte doit être composée uniquement de 6 caractères : **0** pour un espace vide, **1** pour un mur, et **N**, **S**, **E** ou **O** pour la position de départ du joueur et son orientation d'apparition.

Voici un exemple de carte valide :

111111
100101
101001
1100N1

- La **carte** doit être **fermée**/entourée de murs ; dans le cas contraire, le programme doit renvoyer une erreur.
- À l'exception du contenu de la carte, chaque type d'élément peut être séparé par une ou plusieurs lignes vides.
- À l'exception du contenu de la carte, qui doit toujours figurer en dernier, chaque type d'élément peut être placé dans n'importe quel ordre dans le fichier.
- À l'exception de la carte, chaque type d'information d'un élément peut être séparé par un ou plusieurs espaces.
- La carte doit être analysée telle qu'elle apparaît dans le fichier. Les espaces font partie intégrante de la carte et leur gestion est à votre charge. Vous devez être capable d'analyser tout type de carte, tant que celle-ci respecte les règles définies pour la carte.
- À l'exception de la carte, chaque élément doit commencer par son identifiant de type (composé d'un ou deux caractères), suivi de ses informations spécifiques dans un ordre strict :

* Texture nord :

NO ./path_to_the_north_texture

- identifiant : **NO**
- chemin vers la texture nord

* Texture sud :

SO ./path_to_the_south_texture

- identifiant : **SO**
- chemin vers la texture sud

* Texture ouest :

WE ./path_to_the_west_texture

- identifiant : **WE**
- chemin vers la texture ouest

* Texture Est :

EA ./path_to_the_east_texture

- identifiant : **EA**
- chemin vers la texture Est

* Couleur du sol :

F 220,100,0

- identifiant : **F**
- couleurs RVB dans la plage [0,255] : **0, 255, 255**

* Couleur du plafond :

C 225,30,0

- identifiant : **C**
- Couleurs R,V,B dans la plage [0,255] : **0, 255, 255**

- Exemple de la partie obligatoire avec une scène **.cub** minimaliste :

```
NO ./path_to_the_north_texture
SO ./path_to_the_south_texture
WE ./path_to_the_west_texture
EA ./path_to_the_east_texture
F 220,100,0
C 225,30,0
11111111111111111111111111111111
10000000001100000000000001
10110000011100000000000001
10010000000000000000000001
111111110110000011100000000001
10000000001100000111011111111111
1111011111111011100000010001
1111011111111011101010010001
11000000110101011100000010001
100000000000000001100000010001
100000000000000001101010010001
11000001110101011111011110N0111
11110111 1110101 101111010001
11111111 1111111 111111111111
```

- Si une erreur de configuration, quelle qu'elle soit, est rencontrée dans le fichier, le programme doit se terminer correctement et renvoyer « **Error\n** » suivi d'un message d'erreur explicite de votre choix.

Chapitre VI

Exigences relatives au fichier README

Un fichier README.md doit être fourni à la racine de votre dépôt Git. Il a pour but de permettre à toute personne ne connaissant pas le projet (pairs, équipe encadrante, recruteurs, etc.) de comprendre rapidement en quoi consiste le projet, comment le lancer et où trouver des informations complémentaires à ce sujet. Le fichier README.md doit inclure au minimum :

- La toute première ligne doit être en italique et se lire comme suit : ***This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].***
- Une section « **Description** » présentant clairement le projet, notamment son objectif et un bref aperçu.
- Une section « **Instructions** » contenant toutes les informations pertinentes concernant la compilation, l'installation et/ou l'exécution.
- Une section « **Resources** » listant les références classiques relatives au sujet (documentation, articles, tutoriels, etc.), ainsi qu'une description de l'utilisation de l'IA, en précisant les tâches et les parties du projet concernées.

■ **Des sections supplémentaires peuvent être nécessaires selon le projet** (par exemple : exemples d'utilisation, liste des fonctionnalités, choix techniques, etc.).

Votre fichier README doit être rédigé en anglais.